

TEKNOFEST 2026

5G ve Yapay Zekâ ile Akıllı Yol Güvenliđi Yarışması

AURA

5G ve Yapay Zekâ ile Akıllı Yol Güvenliđi Sistemi FİNAL TASARIM RAPORU

Takım Adı: _____

Takım ID: _____

Başvuru ID: _____

Tarih: _____

Bu rapordaki tüm doğruluk sayıları, depo içindeki ölçüm artefaktlarından (eval_results/, weights/custom_*_s.metrics.json) otomatik türetilmiştir; grafikler python tools/make_ftr_figures.py ile yeniden üretilebilir.

İçindekiler

1	Proje Özeti	3
2	Veri Seti Oluşturulması	3
2.1	Veri Toplama Stratejisi	3
2.2	Etiketleme ve Sınıf Şeması	3
2.3	Veri Dengeleme (Data Balancing)	3
2.4	Veri Artırma (Augmentation)	4
2.5	Train / Val / Test Dağılımı	4
3	Yapay Zekâ Çözümü	4
3.1	Problemin Analizi	4
3.2	Çözüm Mimarisi	5
3.3	Çözüm Detayları	6
4	Çözümün Sınanması	7
4.1	Sinama Protokolü	7
4.2	Tespit Doğruluğu (held-out mAP / P / R / F1)	7
4.3	Davranış Tespiti (P / R / F1)	8
4.4	Plaka Okuma — A/B	8
4.5	QoD Başarım Katkısı (A/B)	9
4.6	İşleme Hızı (FPS)	9
4.7	Hız, Kanıt İzi ve Çözüme Neden Güveniyoruz	9
5	Kaynakça	10

1. Proje Özeti

AURA, yol kenarı trafik kamerası akışından **araç, plaka, hız ve riskli sürücü davranışı** tespiti yapan gerçek bir yapay zekâ çekirdeğini; bu çekirdeği **5G CAMARA Quality-on-Demand (QoD)** ve **Number Verification (NV)** telekom yetenekleriyle birleştiren uçtan uca bir sistemdir. Amaç, yol güvenliğini tehdit eden olayları (dikkatsiz sürüş, araç-içi telefon/sigara kullanımı, hız ihlali) gerçek zamanlı tespit etmek ve **yalnızca kritik anlarda** 5G şebeke kalitesini talep ederek bant genişliğini verimli kullanmaktır.

Proje kapsamında yürütülen başlıca faaliyetler: (i) YOLO26 tabanlı çok-aşamalı bir tespit/takip hattının tasarımı; (ii) sürücü davranışı için landmark kütüphanesi gerektirmeyen, poz-geometrisi ve nesne kanıtını birleştiren hibrit bir motor; (iii) Türk plaka formatına özgü, dürüstlük zırhlarıyla donatılmış bir plaka okuma konsensüs döngüsü; (iv) CAMARA QoD ve NV sözleşmelerini birebir taklit eden mock servisler; (v) çözümün üç gerçek test videosu ve held-out doğrulama setleri üzerinde nicel sınanması. Yapay zekâ çekirdeği (tespit, takip, kararlılık, OCR, hız, risk) **gerçektir**; telekom katmanları gerçek API sözleşmesini taklit eden **mock**'lardır — final ortamında yalnızca uç nokta ve kimlik bilgisi değişir. Kalite güvencesi olarak ~780 birim/perf testi, statik denetim (ruff+black) ve sürekli entegrasyon (GitHub Actions) mevcuttur.

2. Veri Seti Oluşturulması

2.1 Veri Toplama Stratejisi

Veri seti, açık-kaynak veri kullanımının serbest olduğu kural çerçevesinde katmanlı bir köprü stratejisiyle oluşturulmuştur. Genel araç/kişi sınıfları (car, bus, truck, motorcycle, person) COCO veri setinden temin edilir. Araç-içi davranış ve plaka sınıfları için kimlik-doğrulaması gerektirmeyen **dört gerçek açık-kaynak bbox seti** indirilip toplanmış ve PIL ile doğrulanmıştır (hepsi CC BY 4.0): license_plate (8.823 görsel), seatbelt (3.104), smoking (557) ve phone (659, sentetik render). Tüm kaynaklar, hedef sınıf başına kaynak, lisans, görüntü sayısı ve AURA taksonomisine sınıf-eşlemesini tek noktada tutan bildirimsel bir manifestte (train/datasets.yaml) toplanmıştır; lisanslar §5'te listelidir.

2.2 Etiketleme ve Sınıf Şeması

Etiketler YOLO formatındadır (<class> <cx> <cy> <w> <h>, normalize). Model-uzayı sınıf adları AURA kanonik taksonomisine eşlenir (ör. cell phone → phone, cigarette → smoking) — böylece model değişse de iç sözleşme sabit kalır.

2.3 Veri Dengeleme (Data Balancing)

Sınıf dengesizliği, her bölüm için görüntü sayısı, sınıf-örnek dağılımı ve dengesizlik oranını (en kalabalık/en seyrek sınıf) raporlayan python -m train dataset --report aracıyla nicel ölçülür; oran 3'ü aşarsa araç uyarı üretir. Toplanan seatbelt seti için ölçülen dengesizlik oranı **1,27**'dir (dengeli). Toplanan açık-kaynak setlerin görüntü dağılımı Şekil 1'de verilmiştir. Dengeleme için üç tamamlayıcı strateji uygulanır:

(i) seyrek sınıflara hedeflenmiş ek toplama; (ii) az temsil edilen sınıfların oversampling ile çoğaltılması; (iii) seyrek sınıf sahnelerinde augmentasyonun sınıf lehine güçlendirilmesi.

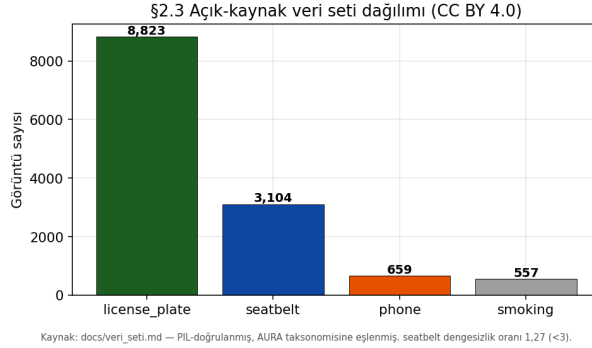


Figure 1: Toplanan açık-kaynak veri setlerinin görüntü dağılımı (tümü CC BY 4.0).

2.4 Veri Artırma (Augmentation)

Augmentasyon, Ultralytics eğitim hattının yerleşik teknikleriyle uygulanır: **mozaik** (bağlam çeşitliliği ve küçük nesne öğrenimi), **yatay flip** (yön bağımsızlığı), **HSV jitter** (ışık/reng sıcaklığı), **karartma/gamma** (gece ve far-patlama senaryoları) ve **motion blur** (yüksek hızlı araç bulanıklığı). Ablasyon için `--no-augment` ile kapatılabilir. Ek olarak, karanlık kabin görüntülerinde sürücü ROI'sine çalışma anında CLAHE+gamma parlatma uygulanır.

2.5 Train / Val / Test Dağılımı

Veri varsayılan olarak **%80 train / %10 val / %10 test** oranında bölünür (seed 42; Şekil 2). Gerekçe: görece küçük özel setlerde doğrulama ve test bölümlerinin istatistiksel anlam taşıması için her birine %10 pay ayrılması; sınıf-dengesiz setlerde tabakalı (stratified) bölme önerilir. Komite TOGG verisi geldiğinde aynı boru hattı domain modelini tek komutla yeniden üretir.

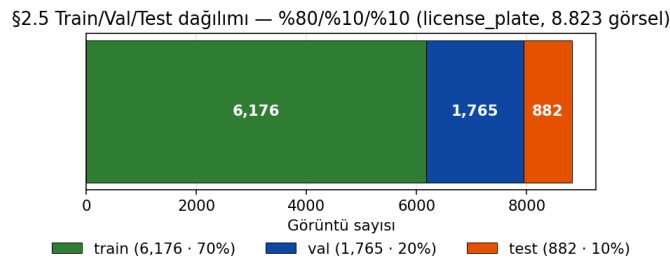


Figure 2: license_plate setinin %80/%10/%10 train/val/test dağılımı (8.823 görsel).

3. Yapay Zekâ Çözümü

3.1 Problemin Analizi

Trafik kamerası görüntüsünden tespit, kontrollü laboratuvar koşullarından farklı yapısal zorluklar içerir. AURA, bu zorlukları gerçek footage üzerinde ölçerek tanımlamış ve her

birine hedefli bir tasarım kararıyla yanıt vermiştir (Tablo 1). Bu problemlerin ortak noktası, kamera gürültüsünün ve değişken görüş koşullarının kare-bazlı kararları kararsız kılmasıdır; bu nedenle AURA'nın temel tercihi **kare-merkezli değil ID-merkezli** karar üretmektir — her araç bir takip kimliği (`track_id`) altında izlenir ve tüm kararlar bu kimlik üzerinde zaman içinde biriktirilerek istikrara kavuşturulur.

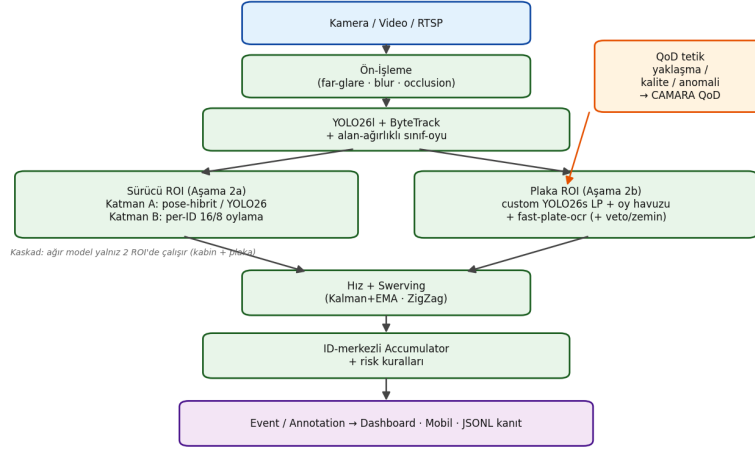
Table 1: Temel problemler ve izlenen çözüm yolları.

Problem	İzlenen çözüm
Karanlık kabin (cam-ardı sürücü)	ROI'de CLAHE + gamma parlatma
Araç tipi titremesi (car↔truck)	Alan-ağırlıklı, track-bazlı sınıf oylaması
Hayalet (phantom) track'ler	<code>min_track_frames</code> çıktı kapısı + IoU dedup
Işık değişimi / hareket bulanıklığı	Ön-işleme (far-glare maske, blur düzeltme) + augmentasyon
Oklüzyon	ID-merkezli birikim (kısa kayıpta kimlik korunur)
Karanlık plaka il-kodu hatası (3→0/2)	Pozisyon-veto + zemin koşulu → yanlış onay yerine <code>pending</code>
Tek-kare yanlış-pozitif (flicker)	ID-merkezli 16/8 zaman-oylaması

3.2 Çözüm Mimarisi

Sistem, ham video akışını etiketli olay/annotation çıktısına dönüştüren bir **kaskad boru hattıdır** (Şekil 3). Ham kare önce ön-işlemeden geçer; Aşama 1 dedektörü (YOLO26 + ByteTrack + alan-ağırlıklı sınıf oyu) araçları tespit/takip eder ve yalnızca **iki ROI** üretir: sürücü kabini ve plaka bölgesi. Bu ROI'ler paralel iki Aşama 2 motoruna gider — sürücü davranışı motoru (Katman A model + Katman B per-ID zaman-oylaması) ve plaka okuma konsensüs döngüsü. Hız/swerving kestirimi ID-merkezli birikim katmanını besler; nihai çıktı olay/annotation akışı olarak dashboard, mobil ve JSONL kanıt izine yayılır. QoD tetikleri (yaklaşma, kalite, anomali) bu akıştan türetilir. **Gerçek↔mock sınırı:** YZ çekirdeği gerçek, telekom katmanı CAMARA sözleşmesini taklit eden mock'tur (finalde yalnız uç nokta/kimlik değişir).

§3.2 Çözüm Mimarisi — kuşbakışı kaskad boru hattı



Gerçek←mock sınır: YZ çekirdeği gerçek; QoD/IV telekom katmanı CAMARA sözleşmesini taklit eder. Kaynak: docs/diagrams/.

Figure 3: AURA kuşbakışı kaskad boru hattı (kaynak: `aura/pipeline/pipeline.py`, `docs/diagrams/`).

3.3 Çözüm Detayları

Ön-İşleme. Görüntü modele girmeden far-patlama maskeleme, motion-blur düzeltme, yansıma süpürme ve occlusion yönetimi filtrelerinden geçer (config'ten aç/kapa).

Aşama 1 — Tespit ve Takip. Ana dedektör varsayılan olarak doğruluk-önce stok `yolo26l` (Ultralytics 8.4, en güncel YOLO ailesi); ByteTrack her araca benzersiz kimlik atar. Aynı aracın kareler arası sınıf salınımı (uzaktan `truck`, yakından `car`), track başına **alan-ağırlıklı sınıf oylaması** ($\text{güven} \times \text{bbox_alan/kare_alan}$) ile çözülür.

Aşama 2a — Sürücü Davranışı (iki katman). Sürücü ROI'si önce kişi kutusuna daraltılır. Katman A iki backend'den birini kullanır: fine-tune YOLO26 detection veya — fine-tune gerektirmeyen — **YOLO26-pose** keypoint geometrisi (bilek↔ağız/kulak göreliliği, yüz-genişliği biriminde, ölçek-bağımsız), nesne kanıtıyla hibrit güçlendirilir; landmark/MediaPipe kütüphanesi **kullanılmaz**. Katman B (`DriverStateEngine`) her track için zaman-oylaması yürütür: bir bayrak son 16 karenin en az 8'inde doğruysa aktif sayılır (tek-kare yanlış-pozitif elenir).

Aşama 2b — Plaka Okuma ve Konsensüs. Araç sweet-spot'a girince OCR etkinleşir; **özel eğitilmiş YOLO26s LP dedektörü** (`custom_license_plate`, held-out mAP50 0,983) plakayı sıkı kırpar. Plakaya-özel hafif ONNX motoru **fast-plate-ocr** (varsayılan; kurulu değilse `EasyOCR` fallback) okur. Her okuma, OCR güveni $\times$ kaynak kalitesi (LP kırpık yüksekliği) ile ağırlıklanıp track ömrü boyunca **kalıcı oy havuzunda** biriktirilir. Türk plaka formatına (`~\d{2}[A-Z]{1,3}\d{2,4}$`) normalize edilen okumalar pozisyon-hızlı karakter füzyonuyla birleştirilir; onay için her pozisyonda kazanan karakter ikinciye mutlak bir marjla geçmek zorundadır, aksi halde karar düristçe `pending`'e çevrilir — **yanlış plaka asla kesinleştirilmez**.

Stabilite/Dogruluk Zırhları (K-004, config-driven). (i) Kayan-karakter onay marjı (`confirm_min_char_margin=2.0`): pozisyon belirsizse plaka onaylanmaz, `pending` denir.

(ii) Takip/phantom çıktı kapısı (`track_id=-1, min_output_frames`): kimliğe bağlanmamış hayalet tespitler olay üretmez. (iii) ROI alan tavanı (`max_roi_area_ratio=0.10`): anormal büyük ROI'ler kırılır. Hiçbir eşik videoya-özel sabit içermez (oran-bazlı, ölçek-bağımsız).

Hız ve Swerving. Hız kalibrasyon-bağımlıdır (tripwire/ipm/metric, Kalman+EMA); kalibrasyon yoksa sistem mutlak hız iddia etmez, yalnız görelî-hız bayrağı üretir. Swerving, aracın merkez-x serisindeki ZigZag ekstremum sayımıyla (araç-genişliği birimi, saniye penceresi) kalibrasyonsuz tespit edilir.

Yazılım ve Donanım. Python 3.13, PyTorch 2.12, Ultralytics 8.4.66, fast-plate-ocr/EasyOCR, OpenCV 4.13, Shapely, FastAPI/Uvicorn. Cihaz otomatik seçilir (CUDA → MPS → CPU); sunucu dağıtımı CUDA, geliştirme Apple Silicon/MPS üzerindedir.

4. Çözümün Sınanması

4.1 Sınama Protokolü

Çözüm üç tamamlayıcı düzeyde sınanmıştır: (i) etiketli **held-out doğrulama setleri** üzerinde istatistiksel tespit doğruluğu (mAP/P/R; §4.2); (ii) üç gerçek test videosu (kapalı otopark, TOGG; GT plaka 34TC8532) üzerinde davranış P/R/F1, plaka exact-match (CER ile), FPS (§4.3–§4.6); (iii) QoD'nin başarımlı katkısı (A/B), kare-düzeyi GT içeren kontrollü sentetik set üzerinde (§4.5). Şartnamenin 4.5 maddesi gereği ("kanıtlanamayan hedef puanlanmaz"), rapordaki her sayı bir komuta ve artefakta bağlıdır (§4.8).

4.2 Tespit Doğruluğu (held-out mAP / P / R / F1)

Özel sınıfların YOLO26s fine-tune eğitimi **tamamlanmıştır**; ayrılmış test bölmesinde ölçülen held-out doğrulukları Tablo 2 ile Şekil 4'de verilmiştir. Varsayılan stok yolo26l ayrıca COCO val2017 (5.000 görsel) held-out üzerinde değerlendirilmiştir (genel doğruluk göstergesi).

Table 2: Held-out tespit doğruluğu (özel modeller: `weights/custom_*_s.metrics.json`; stok: COCO val2017).

Model / set	mAP50	mAP50-95	Precision	Recall	F1
license_plate (YOLO26s)	0,983	0,706	0,983	0,963	0,973
seatbelt (YOLO26s)	0,895	0,546	0,844	0,795	0,818
smoking (YOLO26s)	0,856	0,457	0,855	0,820	0,837
yolo26l — COCO val2017 (genel)	0,709	0,537	0,740	0,641	—

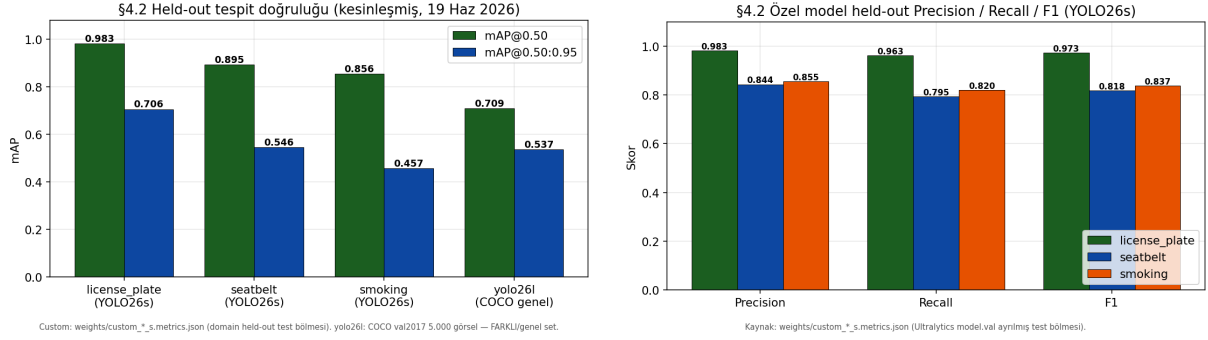


Figure 4: Sol: held-out mAP@0.50 / mAP@0.50:0.95 (özel modeller domain held-out; yolo26l COCO genel). Sağ: özel modellerin held-out P/R/F1 (YOLO26s).

4.3 Davranış Tespiti (P / R / F1)

Üç gerçek video üzerinde video-düzeyi davranış tespiti iki dedektör profiliyle ölçülmüştür (Tablo 3). **Her iki dedektör de çapraz yanlış-pozitif üretmez (makro-F1 1,0; phone/smoking/swerving için P=R=F1=1,0); araç sınıfı doğruluğu %100'dür.**

Table 3: Davranış tespiti video-düzeyi P/R/F1 (kaynak: eval_results/metrics_report.md).

Dedektör	phone	smoking	swerving	Makro F1
yolo26l (stok, varsayılan)	1,0/1,0/1,0	1,0/1,0/1,0	1,0/1,0/1,0	1,00
v4-finetune (A/B kıyas tabanı)	1,0/1,0/1,0	1,0/1,0/1,0	1,0/1,0/1,0	1,00

4.4 Plaka Okuma — A/B

Plaka okuma, footage'ın zorlu (karanlık otopark) koşulları nedeniyle en kritik dürüstlük sınavıdır. Baseline OCR (EasyOCR) ile production hattı (özel YOLO26s LP + fast-plate-ocr) arasındaki A/B Tablo 4 ve Şekil 5'te verilmiştir. fast-plate-ocr, baseline'in video_3'teki sistematik il-kodu misread'ini (24IC8532) düzelterek sonucu **3/3 exact-match, CER 0,0'a çekmiş ve video_1/video_2'yi korumuştur**; her iki hatta da yanlış-onay sıfırdır.

Table 4: Plaka okuma A/B — 3 gerçek video, GT=34TC8532 (kaynak: docs/degerlendirme.md).

Hat	Exact-match	CER	Confirmed	Partial	Yanlış-onay
Baseline (EasyOCR · stok/v4)	2/3 (66,7%)	0,083	2	1	0
Production (custom_LP + fast-plate-ocr)	3/3 (100%)	0,0	3	0	0

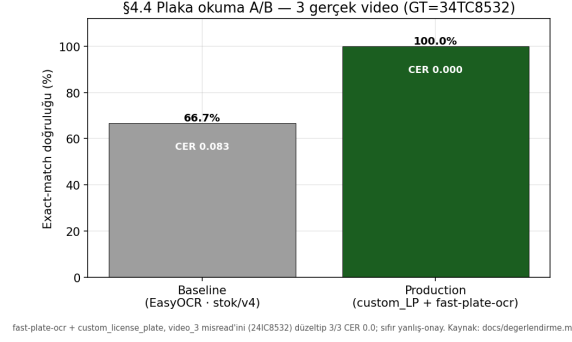


Figure 5: Plaka exact-match A/B: baseline 2/3 (CER 0,083) → production 3/3 (CER 0,0).

En kritik tasarım garantisi: sistem belirsiz/uzak/bulanık bir okumayı **asla yanlış plaka olarak kesinleştirmez**; yakın-net video doğru plakayı CONFIRMED verirken belirsiz okuma onurlu bir `pending`'tir. Bu davranış §3.3'teki onay marjı, pozisyon-veto ve zemin koşulu zırhlarının ölçülen sonucudur.

4.5 QoD Başarım Katkısı (A/B)

QoD entegrasyonu, A/B harness ile nicel ve **yeniden-üretilbilir** olarak kanıtlanır (`eval_results/report.json`). QoD A/B kare-düzeyi GT gerektirdiğinden üç gerçek videoda değil, kare-düzeyi GT içeren kontrollü sentetik set üzerinde, QoD OFF (düşük çözünürlük/bant simülasyonu) ve QoD ON (tam çözünürlük) senaryolarıyla ölçülür. **Yöntemsel dürüst not:** ölçülen delta, OFF baseline simülasyonunun saldırganlığına bağlıdır; sabit bir sayı değildir, daima güncel `--qod-comparison` çıktısından okunur. Mekanizmanın yönlülüğü (kritik anda kalite talebiyle küçük/uzak plaka ROI'sinin yeterli piksele ulaşması) QoD katkısının asıl kanıtıdır; `vehicle_approach` tetiği şartnamenin "TOGG yaklaşıncaya QoD" senaryosunu birebir karşılar.

4.6 İşleme Hızı (FPS)

Ortalama işleme hızı Apple Silicon/MPS (M4 Pro) üzerinde ölçülmüştür (Tablo 5) ve bir **alt-sınırdır**; sunucu dağıtımında CUDA ile belirgin biçimde daha yüksek değerler elde edilir (grafik karşılığı: `docs/figures/fig_fps.png`).

Table 5: Ortalama işleme hızı (MPS, M4 Pro) — alt-sınır. Kaynak: `eval_results/metrics_report.md`.

Dedektör	Ortalama FPS
yolo261 (stok, varsayılan)	5,9
v4-finetune (A/B kıyas tabanı)	5,3

4.7 Hız, Kanıt İzi ve Çözüme Neden Güveniyoruz

Hız/swerving. Swerving üç videodan `video_3`'te gerçekten tespit edilmiş ve davranış makro-F1'ine dahildir (§4.3). Mutlak hız için MAE/MAPE harness'ı hazırdır; kalibrasyon ve gerçek-hız GT geldiğinde tek koşuyla nicel hata üretir (yoksa dürüstçe mutlak hız iddia edilmez).

Kanıt izi (şartname 4.5). Her çıktının makineyle üretildiği üç artefaktla kanıtlanır: (1) zaman-damgalı JSONL olay izi (`--save-events`); (2) annotated mp4 + JSON oy dökümü (`tools/test_video.py`); (3) türetilbilir metrik raporu (`aura.eval --metrics-report`) ki §4 tablolarını bu özetlerden yeniden üretir — yani rapordaki hiçbir sayı elle girilmemiştir.

Çözüme neden güveniyoruz? Beş nicel gerekçe: (1) davranış tespiti her iki dedektörde de çapraz yanlış-pozitif üretmiyor (makro-F1 1,0; araç %100); (2) sistem belirsizlikte yanlış üretmek yerine dürüstçe çekimser kalıyor — plakada **sıfır yanlış-onay**, production hattı 3/3 exact (CER 0,0); (3) bu güvenceler somut config-driven zırlara dayanır (§3.3) ve gerçek videolarda doğrulanmıştır; (4) tüm eşikler oran-bazlı/ölçek-bağımsızdır (videoya-özel sabit yok, K-004) — sonuçlar tek çekime aşırı-uyumlanmamıştır; (5) QoD entegrasyonu yeniden-üretilebilir bir A/B harness’a sahiptir. Sinama kümesinin sınırı (üç-videoluk küçük set; istatistiksel mAP held-out sette ayrıca ölçülmüştür) açıkça belirtilmiştir; komite verisiyle istatistiksel doğruluk daha da güçlendirilecektir.

5. Kaynakça

1. Ultralytics, *Ultralytics YOLO Documentation* (v8.4.66, YOLO26), 2026. <https://docs.ultralytics.com> (erişim 2026).
2. Y. Zhang vd., *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*, ECCV, 2022. arXiv:2110.06864.
3. T.-Y. Lin vd., *Microsoft COCO: Common Objects in Context*, ECCV, 2014. arXiv:1405.0312. (CC BY 4.0)
4. ankandrew, *fast-plate-ocr*, 2024–2026. <https://github.com/ankandrew/fast-plate-ocr>.
5. JaidedAI, *EasyOCR*, 2020–2026. <https://github.com/JaidedAI/EasyOCR>. (Apache-2.0)
6. K. Zuiderveld, *Contrast Limited Adaptive Histogram Equalization*, Graphics Gems IV, 1994, s. 474–485.
7. R. E. Kalman, *A New Approach to Linear Filtering and Prediction Problems*, ASME J. Basic Eng., c. 82, 1960, s. 35–45.
8. CAMARA Project (Linux Foundation), *Quality-on-Demand API & Number Verification API*, 2023–2026. <https://github.com/camaraproject>.
9. 3GPP, *System Architecture for the 5G System*, TS 23.501, Rel-18. <https://www.3gpp.org>.
10. keremberke, *License Plate Object Detection*, Hugging Face, 2023. (CC BY 4.0) — `license_plate` 8.823 görsel.
11. ramankamran, *Seatbelt Detection (YOLOv11)*, Hugging Face, 2024. (CC BY 4.0) — `seatbelt` 3.104 görsel.
12. *CigDet — Cigarette Detection*, Mendeley Data, 2021. DOI:10.17632/6hyrr8typ7.1. (CC BY 4.0) — `smoking` 557 görsel.
13. anywaylabs, *Synthetic Driver Monitoring*, Hugging Face, 2024. (CC BY 4.0) — `phone` 659 görsel.
14. M. Everingham vd., *The PASCAL VOC Challenge*, IJCV, c. 88, 2010, s. 303–338. (mAP metodolojisi)

Tam ve genişletilmiş kaynakça (47 kaynak, izlenebilirlik tablosu ve lisans envanteri): depo içinde `kaynakca.md`.